

PHOTO EDITING WORKFLOW

PhotoLab

User Manual with Developer Annex

Non-destructive RAW development, local adjustments, library management,
composite imaging, GPS mapping, and media tools.

Updated August 2026

Contents

Getting started6

Import from an SD card6

Develop vs Library6

Main interface6

Compare7

Zoom / pan7

Clipping warning7

Metadata7

Filmstrip filters & multi-select7

Compare selected7

Focus peaking7

Star ratings8

Reject / Pick / Color labels8

Undo / Redo / History8

Named snapshots8

Presets / plugin folder8

Light8

Tone curve8

Match across selection9

Color9

Detail9

White balance extras9

Noise reduction9

Sharpening9

Presets9

Output sharpening (PPI)10

Skin protection10

Batch tools10

Import & culling10

Import Photos (Ctrl+Shift+I)10

Culling Mode (F7)10

Library catalog10

Lensfun database	11
Geometry	11
Effects	11
Local adjustments	11
Control points	11
Graduated filter (G)	12
Adjustment brush (Shift+B)	12
HDR	12
Panorama (OpenCV v1)	12
Create Pan Video	12
Focus Stack	13
Library	13
Export and recipes	13
Keyboard shortcuts	14
Tips	15
About	15
Focus stack & panorama	15
Focus stack	15
Panorama	15
HDR merge	15
Panorama to Video	15
Diagnostics	16
Color Calibration Studio	16
Display calibration	16
Camera chart profile	16
Updates & packaging	16
Zone System (Ansel Adams)	16

Help manuals	16
Toolbar	17
Infrared & Astro (Effects tab)	17
Infrared	17
Astro	17
PhotoLab Developer Manual	18
Entry point	18
main.py	18
Core processing	18
imaging.py	18
presets.py	19
qt_utils.py	19
UI	20
main_window.py - PhotoLab(QMainWindow)	20
widgets.py	20
workers.py	22
catalog.py	22
Legacy	22
photo_studio.py	22
Data flow (edit one image)	22
Extending PhotoLab	23
Dependencies	23
GPS metadata and map workflow	23
Color Calibration Studio	23
SD-card import	24
White-balance invariant	24
Release checklist	24
Files checklist	25
Conventions	25

Focus stacking	25
Panorama (OpenCV Stitcher v1)	25
Panorama to Video integration	26
Resource paths & packaging	26
app_paths.py	26
Folders to ship with the executable	26
Tone / color (#6)	26
Detail / WB (#7)	26
Geometry (#8)	27
Lensfun local DB	27
Catalog (#9)	27
Import / culling (#10)	27
Batch (#11)	27
Stack / Pano (#12)	27
HDR (#13)	27
Pan Video (#14)	27
Diagnostics (#15)	28

USER MANUAL

PhotoLab User Manual

Practical guidance for importing, organizing, editing, comparing, mapping, and exporting photographs

PhotoLab is a non-destructive photo editor inspired by applications such as DxO PhotoLab. Edits are stored as a *recipe* and applied on demand-your original files stay untouched.

Getting started

1. Start the app: `python main.py`
2. **File** → **Open Folder for Editing...** (Ctrl+O) to load images into **Develop** mode.
 - **File** → **Recent Folders** reopens folders you used before.
3. Click a thumbnail in the filmstrip to edit it.
4. Use the right-hand panels (**Light**, **Color**, **Detail**, **Geometry**, **Effects**, **Local**) to adjust.
5. Export when ready: **File** → **Export to Disk...** (Ctrl+E).

Open Folder only fills the Develop filmstrip. It does **not** add photos to the Library catalog.

Import from an SD card

Choose **File** → **Import from SD Card...**, then choose the card/camera folder as the source and a normal local folder as the destination. PhotoLab discovers supported photos and videos, skips identical files already copied, gives safe unique names to collisions, and can preserve the card's folder structure. The progress dialog can cancel the remaining copies. Verify the import and backup before formatting a card.

Develop vs Library

Mode	Shortcut	Purpose
Develop	Ctrl+D	Edit the current folder's images
Library	Ctrl+L	Browse a scanned catalog by date, rate, keyword, search

- **Scan Folder into Library...** (Ctrl+Shift+O) recursively indexes a folder tree into a local SQLite catalog.
- Double-click a library thumbnail to open it in Develop.

Main interface

- **Left:** histogram, navigator (click/drag to pan), history
- **Center:** image canvas (zoom with mouse wheel, pan with Space+drag or middle mouse)
- **Right:** tool categories

- **Bottom (Develop):** filmstrip of the open folder

Compare

- **C** - before/after split
- **B** - side-by-side (toolbar may also offer compare modes)
- **Hold \ or ` ``** - temporary before view; release to restore

Zoom / pan

- Mouse wheel: zoom
- Space + drag or middle-button drag: pan
- Fit / 100% controls when available in the toolbar

Clipping warning

- **J** or **View** → **Clipping Warning**
- **Blue** = crushed shadows · **Red** = blown highlights
- Toggle off with **J** again

Metadata

- **I** or **View** → **Image Metadata...** - path, size, camera, lens, exposure, dates, and other tags

The Metadata panel also shows GPS availability:

- A **green dot** means valid coordinates are present; **Map this photo** opens that location.
- A **red dot** means no coordinates were found.
- **Select GPS photos & show map** scans the filmstrip, selects geotagged images, and opens the map list.
- The browser map uses Leaflet/OpenStreetMap and needs internet access for map tiles.

RAW metadata is read independently of sensor decoding when possible, so camera and GPS fields may remain available even when a new RAW compression requires a newer LibRaw build.

Filmstrip filters & multi-select

- Under the filmstrip: **Filmstrip min** ★ hides lower-rated images (All, 1+, ... 5+)
- **Color** filter: All / None / Red / Yellow / Green / Blue / Purple
- **Ctrl/Shift+click** to multi-select thumbnails
- Keys **0–5**, **X** (reject), **U** (pick) apply to **all selected** images (or the current one if nothing is multi-selected)
- **Ctrl+Shift+1...5** set a color label; **Ctrl+Shift+0** clears it
- Filmstrip bar buttons: ★ **Rate**, ✓ **Pick**, ■ **Reject**, **Compare**
- **Left / Right** arrows skip images hidden by the current filters

Compare selected

- Multi-select 2–4 filmstrip images → **Image** → **Compare Selected...** (**Ctrl+Shift+B**) or the **Compare** button
- Side-by-side preview (with current recipes applied when available); open any frame in Develop from the dialog

Focus peaking

- **P** or **View** → **Focus Peaking**
- Green edge overlay highlights sharp regions (useful for checking focus)

Star ratings

- Keys **0–5** set a star rating on **selected** images (or current); 0 clears
- Shown on the filmstrip label; also written to the Library catalog when that image is indexed

Reject / Pick / Color labels

- **X** - reject / unreject selected (■ on filmstrip); synced to Library when indexed
- **U** - pick / unpick selected (✓ on filmstrip) for keepers
- **Image** → **Color Label** or **Ctrl+Shift+0–5** - red / yellow / green / blue / purple (or clear)

Undo / Redo / History

- **Ctrl+Z** / **Ctrl+Y** step through the History panel
- Click any history entry to jump to that state
- **Right-click** a history entry:
- **Preview before / after...** - side-by-side of that state vs the current recipe
- **Copy settings from this entry** - then **Ctrl+Shift+V** to paste onto the current image
- **Ctrl+R** - reset entire image
- **Edit** → **Reset Module** - reset Tone, Color, Detail, Geometry, Local, or Effects only

Named snapshots

- **Edit** → **Save Snapshot...** - stores a named recipe checkpoint
- Snapshots are **persisted** in the image's `.photoLab.json` sidecar and reload when you open the image again
- **Edit** → **Restore Snapshot...** - pick a named checkpoint to apply

Presets / plugin folder

- **File** → **Load Preset...** / **Save Preset...** default to the app's `plugin/` folder
- Drop `.json` (PhotoLab) or `.xmp` (Lightroom / Camera Raw) presets into `plugin/`
- User override: `~/photoLab/plugin/`
- Sample presets: `neutral.json`, `vivid.json`

Light

Global exposure and tone:

- Exposure, Smart Lighting, Contrast
- Highlights, Shadows, Whites, Blacks
- Clarity, Microcontrast, Gamma
- Tone curve points (when shown)

Use **Auto Exposure** from the Image menu for a starting point.

Tone curve

- **Parametric** - five region handles (shadows → highlights)
- **Luma / R / G / B** - free point curves; double-click to add a point, double-click a point to remove
- **Reset** clears the active curve channel

Match across selection

- **Image** → **Match Exposure to Current** - align median luminance of selected filmstrip images to the current one
 - **Image** → **Match White Balance to Current** - copy temperature / tint to the selection
-

Color

- **White Balance:** As Shot, Temperature, Tint
 - **W - White Balance Picker:** click a neutral gray/white in the image
 - Vibrance / Saturation
 - Selective HSL per color family (hue, saturation, luminance)
 - **Split Toning** (Color panel): independent shadow / highlight hue & saturation, plus balance
 - **Soft Proofing** (Color panel):
 - Enable soft proof and choose a profile: **sRGB**, **DisplayP3**, **AdobeRGB**, **CMYK**, or **Gray**
 - **Rendering intent:** Relative / Perceptual / Saturation / Absolute
 - **Load ICC...** - use a custom .icc / .icm profile (Pillow ImageCms); otherwise PhotoLab tries a system profile, then an approximation
 - **Gamut warning (magenta)** highlights colors that change strongly under the simulation
 - **Simulate paper white** - slight warm paper tint after conversion
 - **Gamut shift %** readout shows how much of the image moved under the proof (and whether ICC or approximate was used)
 - Useful for relative checks; not a substitute for press certification
-

Detail

White balance extras

- **Dual illuminant** - blend primary and secondary temperature/tint (mixed lighting)
- **Mix toward Temp 2** - 0% primary only, 100% secondary only

Noise reduction

- **Luminance** - grain / luminance noise
- **Chrominance** - color blotches
- **Strength** - biases toward stronger NLM denoise
- **Detail Recovery** - restores fine structure after NR (edge-aware)
- **Method:** Auto · Bilateral (fast) · NLM (stronger, slower)

Sharpening

- **Capture sharpening:** Amount, Radius, Masking/Threshold, Detail
- **Output sharpening:** final amount for screen/print

Presets

Light NR, Strong NR, Portrait, Landscape - one-click starting points. Always check at **100% zoom**.

Output sharpening (PPI)

- Set **Output PPI** and **Media** (Screen / Matte / Glossy)
- **Apply suggestion** fills Output amount from PPI + media
- Quick buttons: Screen 96, Print 240 / 300 / 360

Skin protection

- **Protect skin tones** reduces capture/output sharpening and vibrance on skin-like hues
- Portrait detail preset enables moderate skin protection

Batch tools

- **Apply Preset to Selected...** - load one JSON/XMP preset onto all selected filmstrip/library images (writes sidecars)
- **Batch Rename / Move...** - rename patterns (keep, date+seq, date+orig), optional destination folder; **catalog paths** update
- **Export queue** (survives restart; stored in `~/ .photolab/export_queue.json`):
- **Add Selected to Export Queue**
- **View Export Queue...**
- **Process Export Queue...**

Import also accepts an optional **default preset** applied to every imported file.

Import & culling

Import Photos (Ctrl+Shift+I)

- Choose **source** and **destination** folders
- **Copy** or **Move**
- Rename: keep names, YYYYMMDD_0001, or YYYYMMDD_original
- Optional **Year / YYYY-MM-DD** subfolders
- Optionally scan into Library and open the destination when done

Culling Mode (F7)

Full-screen review of the filmstrip (or library selection):

Key	Action
← → / Space	Previous / next
0–5	Rating
X	Reject toggle
U	Pick toggle
Esc	Exit

Library catalog

- **Collections** - create, add selected photos, browse members

- **People / faces** - comma-separated person tags (saved on selected thumbs)
- **Find duplicates** - groups images with the same content fingerprint (built at scan)
- **Virtual copy** - stores a named recipe clone of the current Develop image in the catalog

Lensfun database

Place the Lensfun XML database next to the app, for example:

- photo_lab/lensfun/
- photo_lab/lensfun/data/db/
- photo_lab/lensfun/version_1/

Also: `pip install lensfunpy`. **Test match...** in Geometry shows which DB path was used.

Geometry

- Horizon angle, Distortion, Perspective
- **Level horizon (L)**: drag a line along the real horizon; angle is applied for you
- Or use the **Level: draw line on image** button under Geometry → Horizon
- **Auto level** uses Hough edge detection as a starting suggestion
- **Keystone (4 corners)** lets you drag TL/TR/BR/BL handles directly on the image
- **Show grid** - rule of thirds + center crosshair
- **Fibonacci / golden spiral** - composition guide
- Drag center handle to move
- Drag blue corner to resize
- Size slider and Orientation (A–D + mirrored)
- **Guide color** - Yellow, **White**, Cyan, or Black (easier to see on busy images)
- Crop tool with aspect presets: Free, Original, 1:1, 5:4, 4:3, 3:2, 16:9, 16:10, portraits, A4
- **Optics / Lens**: chromatic aberration; optional Lensfun auto-correct if lensfunpy is installed

Effects

- ClearView-style dehaze, vignette, film grain
- Black & white
- HDR Look (single-image HDR-style tone mapping)
- Rotate 90°

Local adjustments

Control points

Enable **Local Adjustments**, click the image to place a point, then refine:

- Size, Feather
- **Chroma range / Luma similarity** - limit the effect to similar colors/tones at the point
- **Luma min / Luma max** - absolute luminance range (only tones in this band are affected)
- Exposure, Contrast, Saturation, Clarity

Graduated filter (G)

1. Press **G** or use the toolbar.
2. Drag on the image to place a linear gradient.
3. Drag end handles to reshape.
4. Adjust exposure, contrast, saturation, clarity, temp, feather in the Local panel.

Adjustment brush (Shift+B)

1. Press **Shift+B**.
2. Paint on the image (size / hardness / **flow** / **opacity** in Local panel).
3. **Mask mode**: Add · Subtract · Intersect (or Eraser / right-drag).
4. **Invert selected mask** - apply adjustments outside the painted area.
5. **Auto subject mask** - offline OpenCV GrabCut (no neural net); creates a new brush entry you can refine.
6. Show mask only to preview coverage.

HDR

- **Effects** → **HDR Look** for a single photo.
- Multi-exposure: select 2+ filmstrip images → **Image** → **Merge HDR...** (Ctrl+Shift+H).

Panorama (OpenCV v1)

Stitch overlapping frames into a wide image:

1. Multi-select ordered frames in the filmstrip (or Library) - ideally **left** → **right**.
2. **Image** → **Panorama...** (**Ctrl+Shift+P**), or **Panorama...** in the Library toolbar.
3. Choose mode (**Panorama** / Scans / Auto) and optional working size.
4. Save (TIFF recommended). The result opens in **Develop**.

Expectations (important):

- OpenCV automatic stitch works best with **~25–40% overlap**, steady exposure/WB, and **little parallax**.
- Moving subjects, strong foreground parallax, or sparse overlap often fail or show seams.
- This is **not** a PTGui/Hugin replacement; quality will improve in later versions.

Tips: tripod or nodal head, manual exposure, consistent focal length, orderly capture.

Create Pan Video

Turn a panorama (or any wide still) into a motion video with the full **Panorama to Video** tool:

1. Open the image in Develop (often the result of **Panorama...**).
2. **Image** → **Create Pan Video...**
3. The companion app opens with your image and output folder already set.
4. Use Ken Burns, cinematic looks, film effects, audio, ffmpeg encoders - **all original features**.

Optional: **Image** → **Audio Editor...** to prepare a soundtrack (cut / fade / normalize) before attaching it in Pan Video.

Requires `pano_video.py` (and optionally `audio_editor.py`) next to PhotoLab, plus **ffmpeg** on PATH for encoding.

Focus Stack

Combine a near-to-far (or far-to-near) focus bracket into one sharp image:

1. Open a folder of focus frames (or multi-select in the filmstrip).
2. **Image** → **Focus Stack...** (**Ctrl+Shift+F**).
3. Choose:
 - **Alignment:** ECC Affine (default), Translation, Rigid, Homography, or ORB variants
 - **Fusion:** Depth map, Weighted, Pyramid, or Average
 - **Reference frame:** middle / first / last
 - Optional working size, common-area crop, depth-map PNG
4. Save the result (TIFF recommended). It opens in **Develop** for further edits.

From **Library**, multi-select brackets and use the **Focus Stack...** toolbar button (same dialog).

Tips: use a tripod, manual exposure/WB, and modest focus steps. Large viewpoint changes may need ORB Homography; expect soft scores on very hard frames.

Library

1. **Scan Folder into Library...**
2. Browse the **date tree** or view all.
3. Rate / reject photos.
4. Search by filename, keywords, camera, folder.
5. Select photos → enter keywords → **Save KW**.
6. **Smart collection** dropdown: All, Picked, Rejected only, Rated 3+, Rated 5, Unrated.
7. **Focus Stack...** in the library toolbar stacks the selected photos.
6. **Remove from Library** - drops catalog entries only (files stay on disk).
7. **Move to Trash** - sends files to the system trash (or a `.photoLab_trash` folder if OS trash is unavailable) and removes them from the catalog. Confirm the dialog carefully.

Optional: `pip install send2trash` for better OS recycle-bin integration.

Export and recipes

Edits are non-destructive **recipes**.

- **Ctrl+S** - save a `.photoLab.json` sidecar next to the file
- Sidecars **auto-load** when you open that image again
- **File** → **Auto-Save Sidecars** - writes the sidecar after each history step while editing
- **Export** options: watermark text, long-edge size, save sidecar
- **Export presets:** Web (2048px JPEG 85), Print (full JPEG 95), Archival (TIFF)
- **Copy / Paste Settings** (**Ctrl+Shift+C / V**) - paste everything or only Tone / Color / Detail / Geometry / Local / Effects
- **Sync Settings to Selected...** (**Ctrl+Shift+S**) - copy the current recipe onto other filmstrip selections
- **Save / Restore Snapshot** - named checkpoints for the current image (separate from linear history)

Batch Export Selected... (**Ctrl+Shift+E**): multi-select filmstrip thumbnails, choose output folder.

Keyboard shortcuts

Key	Action
Ctrl+O	Open folder (Develop)
Ctrl+L / Ctrl+D	Library / Develop
Ctrl+E	Export
Ctrl+Shift+E	Batch export
Ctrl+S	Save recipe sidecar
Ctrl+Shift+D	Debug console
Ctrl+Shift+H	Merge HDR
Ctrl+Shift+F	Focus Stack
Ctrl+Shift+O	Scan into Library
W	White balance picker
G	Graduated filter
Shift+B	Adjustment brush
C	Split compare
F1	User manual (Help)
J	Clipping warning
P	Focus peaking
L	Level horizon (draw line)
I	Image metadata
F11	Full screen
Ctrl+Shift+S	Sync settings to selected
0–5	Star rating (selected or current)
X	Reject / unreject (selected or current)
U	Pick / unpick (selected or current)
Ctrl+Shift+0–5	Color label clear / R / Y / G / B / Purple
Ctrl+Shift+B	Compare selected filmstrip images
Left / Right	Prev / next visible filmstrip image
Ctrl+R	Reset image
Ctrl+Z / Ctrl+Y	Undo / Redo
Ctrl+Shift+C / V	Copy / paste settings

Tips

- Library and filmstrip thumbnails use **embedded RAW previews** when available (much faster than full decode).
 - Judge noise and sharpening at **100% zoom**.
 - Prefer original camera RAW files; partial “Copy” NEFs often fail to decode.
 - Library and Develop are separate workflows-scan only when you want catalog features.
 - Use the **Debug Console** (View menu) if something misbehaves.
 - If a RAW sensor decode is unsupported, PhotoLab may open the full embedded JPEG and labels the fallback clearly. It remains editable, but it does not provide true RAW highlight recovery or 16-bit sensor latitude.
 - DNG is an openly documented container, but vendor-specific compression and tags can still require a newer rawpy/LibRaw version.
-

About

PhotoLab is an open, Python-based editor. It aims for a DxO-like workflow without claiming full parity with commercial products (e.g. DeepPRIME-class denoise or proprietary lens modules).

Focus stack & panorama

Focus stack

- Alignment modes (ECC / ORB) and fusion (depth / weighted / pyramid / average)
- **Min align score** - frames below the threshold are excluded from fusion (reference always kept)
- After completion: **report** with per-frame confidence and **Match style of source frame**

Panorama

- Modes: spherical-like (panoramas) / planar scans / auto
- **Match exposure / WB** before stitch
- **Order by capture time** helper
- Report notes projection behavior; optional match style from first source frame

HDR merge

- **Detect brackets from open folder** - groups by EXIF exposure bias / shutter
- **Mertens** (default exposure fusion) or **Debevec + tonemap** (Reinhard / Drago / Mantiuk)
- **Deghost strength** - blends moving regions toward the stack median before merge
- Align frames for handheld brackets

Panorama to Video

- Dark UI aligned with PhotoLab Develop
- Launched from **Image** → **Create Pan Video...** with a **graded still** baked from the current Develop recipe
- Progress shows **percent + ETA**
- **Refresh Still Preview** re-renders the canvas still without a full encode
- Live preview, 3s test clip, and full render unchanged

Diagnostics

- **Help** → **Report a Problem...** (also on the Debug Console) saves a text report with system info, the last 50 log-file lines, and the Debug Console buffer. The report is also copied to the clipboard.
- RAW failures show clearer messages (partial file, unsupported compression, camera-specific tips) instead of a bare path error.

Color Calibration Studio

Open **Tools** → **Color Calibration Studio...** for measurement-based color profiling. PhotoLab uses the external, open-source ArgyllCMS tools; it does not attempt visual software-only calibration.

Display calibration

1. Install ArgyllCMS and choose its `bin` folder if PhotoLab does not find it automatically.
2. Connect a supported colorimeter or spectrophotometer and warm the display for about 30 minutes.
3. Disable Night Light, HDR, automatic brightness, and competing calibration loaders.
4. Choose the display, white point, brightness, response curve, quality, and output name.
5. Select **Start guided display calibration...** and follow Argyll's measured prompts.
6. In **ICC profiles**, validate the result and optionally install it for the selected display. Installation requires confirmation because it changes the operating-system profile.

For a typical dim-to-moderate editing room, D65, gamma 2.2, and about 100-120 cd/m² are reasonable starting targets. Print-matching environments may use D50 and brightness chosen to match controlled viewing light.

Camera chart profile

Use an evenly lit, glare-free photograph of a supported color chart. Provide the Argyll `.cht` recognition layout and corresponding reference-values file. A neutral TIFF converted from RAW with automatic and creative corrections disabled is preferred. **Build camera ICC profile** runs `scanin` followed by `colprof`; review the Delta E fit report in the log before using the result.

Camera profiles are specific to the camera response and lighting spectrum. Create separate profiles for materially different illuminants when accuracy matters.

Updates & packaging

- **Help** → **Check for Updates...** opens GitHub Releases.
- Portable builds: see **PACKAGING.md** / `build_portable.bat`.
- Pan Video needs **ffmpeg** on PATH.

Zone System (Ansel Adams)

Effects → **Black & White** includes zone mapping: placement (Zone V ≈ mid-gray), expansion (N-/N+), spectral filters, snap, and false-color overlay.

Help manuals

Help → **User Manual** (F1) and **Developer Manual** load Markdown from the `docs/` folder next to the app (or the path set in Preferences).

Toolbar

Button	Action
Open / Scan	Open folder / scan into Library
Library / Develop	Switch modes
Edit / Split / Side-by-Side	Compare views
Fit / 1:1	Fit window / actual pixels (Ctrl+1)
Prev / Next	Navigate filmstrip
Local / Grad / Brush / WB	Tool modes (exclusive)
Reset / Preset / Save Preset	Recipe
Tools ■	HDR, stack, pano, pan video, map, slideshow, print, scripts
Export	Export current image

Number keys **0–5** set star rating (so 1:1 zoom uses **Ctrl+1**).

Infrared & Astro (Effects tab)

Infrared

- **Channel swap R↔B** - classic false-color IR starting point
- **False color** - blend toward a warm foliage / cool sky remix
- **Mono IR** - red-weighted black & white
- Preset buttons: Wood effect, Gold/Blue, Mono IR

Astro

- **Stretch (asinh)** - lift faint nebula / Milky Way structure
- **Background / gradient remove** - soften sky gradients and light pollution
- **Star emphasis** - mild detail boost on point sources
- Presets: Milky Way, DSO soft

These are stored in the recipe sidecar like any other edit. Full calibration stacking (darks/flats) can be added later.

DEVELOPER ANNEX

Architecture, Extension & Release Guide

Implementation notes for maintainers and contributors

PhotoLab Developer Manual

Architecture overview for contributors. PhotoLab is a **PyQt6** desktop app with a **NumPy/OpenCV** processing pipeline and optional **rawpy** for RAW files.

Entry point

`main.py`

- Creates `QApplication`, forces a valid default font (avoids `QFont::setPointSize(-1)` on Windows).
- Sets OpenCV log level to reduce TIFF noise.
- Installs `sys.excepthook` for visible tracebacks.
- Instantiates PhotoLab from `main_window.py` and runs the event loop.

Core processing

`imaging.py`

Central **non-destructive** image pipeline. No Qt imports.

Symbol	Role
<code>Recipe</code>	Dataclass of all edit parameters for one image
<code>Recipe.soft_proof*</code>	<code>soft_proof</code> , <code>soft_proof_profile</code> , <code>soft_proof_gamut</code>
<code>load_image</code>	Loads JPEG/PNG/TIFF via OpenCV; RAW via rawpy under <code>_rawpy_lock</code>
<code>is_raw / RAW_EXTS / IMAGE_EXTS</code>	Extension helpers
<code>_silent_imread</code>	<code>cv2.imread</code> with <code>stderr</code> silenced (TIFF warnings)
<code>extract_exif</code>	Best-effort EXIF (Pillow) including <code>datetime</code> , <code>camera</code> , <code>focal</code> , <code>aperture</code>

Symbol	Role
<code>apply_recipe</code>	Full pipeline: WB → tone → HSL → local → gradients → brushes → optics → denoise → geometry → sharpen → effects
<code>apply_denoise</code>	LAB bilateral / NLM + detail recovery
<code>apply_sharpen / apply_output_sharpen</code>	Edge-masked USM + fine detail + output stage
<code>apply_local_points</code>	Radial control-point masks
<code>apply_gradients</code>	Linear graduated filters
<code>apply_brush_masks</code>	Painted dab masks
<code>apply_chromatic_aberration_fix</code>	Simple radial R/B shift
<code>try_lensfun_correct</code>	Optional lensfunpy geometry correction
<code>merge_hdr_mertens</code>	Multi-exposure fusion (AlignMTB + MergeMertens)
<code>apply_hdr_look</code>	Single-image HDR-style tone mapping
<code>recipe_to_dict / recipe_from_dict</code>	JSON serialization
<code>save_recipe_sidecar / load_recipe_sidecar</code>	<code>.photolab.json</code> next to the image
<code>apply_watermark</code>	Text watermark on export
<code>extract_embedded_preview</code>	Fast RAW thumb via embedded JPEG / half-size decode
<code>apply_soft_proof</code>	ICC soft-proof (ImageCms) + approximate fallback; paper white; gamut %
<code>soft_proof_gamut_percent</code>	Fraction of pixels shifted under proof

Thread safety: All rawpy/LibRaw use goes through `_rawpy_lock`.

Pipeline order (simplified): load → exposure/tone → WB → HSL → soft proof (optional) → control points → gradients → brushes (supports invert) → Lensfun/CA → denoise → geometry (horizon, distortion, crop) → clarity/microcontrast/HDR look → capture sharpen → output sharpen → vignette/grain/B&W.

Brush masks: each entry is `{strokes, hardness, exposure, ..., invert?}`. When `invert` is true, the painted region is the *protected* area and adjustments apply outside it.

Thumbnails: `ThumbnailWorker` and `CatalogThumbWorker` call `extract_embedded_preview` first (embedded JPEG from RAW, else half-size postprocess, else downsampled image).

presets.py

- `xmp_to_recipe` - parse Lightroom-style XMP into a Recipe (regex / ElementTree).
- `load_preset_file` - JSON or XMP presets.

qt_utils.py

- `cv_to_qpixmap` - BGR NumPy array → QPixmap for display.

UI

main_window.py - PhotoLab(QMainWindow)

Main application shell.

Area	Responsibility
Menus / toolbar	File, Edit, View, Image, Help; mode switches; tools
mode_stack	Library page vs Develop page
Develop layout	Left tools, center ImageCanvas, right category tabs, filmstrip
Library page	Date tree, search, keywords, rating, thumbnail grid
recipes	dict[path, Recipe] - per-image edit state
meta_cache	EXIF / RAW metadata per path
render_preview	Debounce (QTimer) apply_recipe → canvas
History	Simple undo stack of recipes
Help	Opens user/developer manuals in a dialog

Important methods

- open_folder / open_folder_path - Develop-only folder load
- scan_library_folder - catalog scan worker
- toggle_wb_picker / toggle_gradient_mode / toggle_brush_mode
- export_current / batch_export_selected
- save_sidecar / reload_sidecar
- _show_user_manual / _show_developer_manual

Category tabs: Light, Color, Detail, Geometry, Effects, Local (control points, gradients, brushes).

widgets.py

Custom Qt widgets:

Widget	Role
ImageCanvas	Zoom/pan, compare modes, crop, control points, spiral, gradients, brush, WB sample
HistogramWidget	RGB/luma histogram
NavigatorWidget	Overview + viewport rectangle
ToneCurveWidget	Curve editor
SliderRow	Labeled slider + spinbox
HistoryWidget	History list UI
ColorWheelWidget	Optional color UI

Canvas signals: `controlPoint*`, `gradientChanged`, `wbPicked`, `brushStrokeFinished`, `brushMaskChanged`.

Clipping overlay: `ImageCanvas.show_clipping` scans the displayed pixmap and paints blue/red blocks for shadow/highlight clipping (J key).

Ratings / flags / color labels: `rate_current`, `toggle_reject_current`, `toggle_pick_current`, and `set_color_label` operate on **all selected filmstrip paths** (via `_target_filmstrip_paths`), falling back to the current image. Labels are composed by `_filmstrip_label_text` / `_refresh_filmstrip_item` (pick ✓, reject ■, color emoji, stars). Color labels live in `_color_labels` (in-memory; not yet a catalog column).

Filmstrip navigation: `prev_image` / `next_image` walk `_visible_filmstrip_paths()` so rating/color filters are respected.

Compare selected: `compare_selected_images` opens a dialog with 2–4 selected frames (recipes applied when present).

Auto-save: `autosave_sidecars` triggers `save_recipe_sidecar` from `_push_history`.

Selective paste: `paste_settings` dialog maps checkbox groups to Recipe field subsets.

History / Undo: `HistoryWidget` stores (`label`, `recipe_dict`) snapshots, truncates the redo branch on new push, and exposes `undo_index` / `redo_index`. Right-click emits `previewRequested` / `copySettingsRequested`. `PhotoLab._on_history_preview` shows a side-by-side dialog; `_on_history_copy_settings` fills `_copied_recipe`.

Reject: `toggle_reject_current` uses `catalog.set_reject` + in-memory `_reject_flags` and updates filmstrip labels (multi-select aware).

Canvas zoom: `mouseDoubleClickEvent` toggles fit vs 1:1.

Horizon line tool: `horizon_line_mode` + drag; emits `horizonLineFinished(angle)` so `PhotoLab._on_horizon_line` sets `Recipe.horizon`.

Metadata dialog: `show_metadata` merges `meta_cache` with `extract_exif`.

Filmstrip filter: `_apply_filmstrip_filter` toggles `QListWidgetItem.isHidden` from ratings.

Focus peaking: `ImageCanvas.show_peaking` samples the displayed pixmap and draws green points on high local contrast edges (P key).

Recent folders: stored in `~/ .photolab_recent.json`; `PhotoLab._add_recent_folder` updates the File menu submenu.

Snapshots: `_snapshots[path] = [{name, recipe, ts}]` via Edit → Save/Restore Snapshot. Persisted in `.photolab.json` under "snapshots"; reloaded in `_on_image_loaded` via `load_snapshots_sidecar`.

Sync to selected: `sync_settings_to_selected` clones `recipes[current]` onto other selected filmstrip paths.

Module reset: `reset_module(which)` restores default Recipe fields for tone/color/detail/geometry/local/effects groups.

Pick flags: `_pick_flags` in-memory; filmstrip label prefix ✓.

Smart collections: `_lib_smart_match` filters catalog rows by picked/rejected/rating modes from `lib_smart_combo`.

Temp before: hold Backslash/backtick sets split compare; release restores previous mode.

Export presets: web/print/archival map to `max_dim` + jpeg quality + optional extension change.

Library export: `_lib_export_selected` builds `BatchExportWorker` jobs, loading sidecars when no in-memory recipe exists.

Navigator viewport: `_update_navigator_viewport` maps canvas offset/scale to normalized rect; `zoom_changed` keeps the yellow view rectangle in sync.

Composition guides: `show_grid`, `interactive show_spiral` (normalized center/scale/orient). `guide_color` (yellow|white|cyan|black) via `set_guide_color`; `_guide_colors()` supplies pens for grid and spiral.

Library trash: `_lib_move_to_trash` uses `send2trash` when installed, else moves files into a sibling `.photolab_trash` folder; always calls `catalog.remove_image`. `_lib_remove_from_catalog` is catalog-only.

workers.py

QThread workers so the UI stays responsive:

Worker	Job
ThumbnailWorker	Filmstrip thumbs (never <code>cv2.imread</code> on RAW)
LoadImageWorker	Background full-res load
ExportWorker	Single export + optional watermark / resize
BatchExportWorker	Multi-file export with per-image recipes
FocusStackWorker	Align + fuse focus brackets → new file
PanoramaWorker	OpenCV stitch → new file
HdrMergeWorker	Bracket merge
CatalogScanWorker	Recursive library scan
CatalogThumbWorker	Library grid thumbs + disk cache

catalog.py

SQLite library:

- Table `images`: path, dates, rating, reject, keywords, camera, folder, etc.
- `scan_folder` - incremental recursive scan; EXIF date or file mtime
- `set_rating` / `set_reject` / `set_keywords` / `remove_image`
- `search` - filename, keywords, camera, folder
- Thumb cache paths under a cache directory

Legacy

photo_studio.py

Older, simpler editor shell. Kept for reference; `main.py` **uses** `main_window.PhotoLab`.

Data flow (edit one image)

```
Open folder → image_paths + filmstrip
→ load_image (worker) → meta_cache + display pixmap
→ Recipe (new or sidecar)
User moves slider → recipes[path].field = value → render_timer
→ apply_recipe(preview_src, recipe, meta=...) → ImageCanvas
Export → ExportWorker → load full → apply_recipe → watermark → imwrite
```

Extending PhotoLab

1. New global adjustment

- Add field on Recipe
- Implement pure function in `imaging.py`
- Call it from `apply_recipe`
- Add slider in the appropriate `_build_*_tab`

2. New local tool

- Store geometry + params on Recipe (list of dicts)
- Draw/interact in `ImageCanvas`
- Apply mask in `imaging.py`

3. New background job

- Add a `QThread` in `workers.py`
- Emit progress/finished signals to PhotoLab

4. Keep `imaging.py` free of Qt so batch and tests stay simple.

Dependencies

- **Required:** Python 3.10+, PyQt6, NumPy, OpenCV, Pillow
- **RAW:** rawpy
- **RAW metadata fallback:** ExifRead (recommended for NEF and other containers Pillow cannot open)
- **Optional:** lensfunpy (optics auto-correct), send2trash, pygame, ffmpeg

GPS metadata and map workflow

`imaging.extract_exif` first uses Pillow and then performs a best-effort ExifRead pass when GPS was not found. This fallback reads metadata IFDs without decoding sensor pixels. `extract_gps` normalizes DMS ratios to signed decimal degrees. Keep metadata extraction independent of RAW decode success.

The Metadata panel updates its red/green GPS indicator from `meta_cache`. The bulk action scans filmstrip paths, selects only geotagged items, and passes those paths to `MapDialog.map_view.py` writes a temporary Leaflet document; map tiles are loaded from OpenStreetMap, while PhotoLab does not upload the image files.

Color Calibration Studio

`color_calibration.py` is a PyQt wrapper around ArgyllCMS, not a replacement color engine. Pure helpers locate executables, build deterministic argument lists, and validate ICC files with Pillow ImageCms.

`ColorCalibrationDialog` exposes:

- display profiling through interactive `dispcal -o`, with display, white-point, luminance, gamma, and quality targets;
- camera chart recognition through `scanin`, followed by matrix/shaper input-profile creation through `colprof`;
- ICC validation and confirmed operating-system installation through `dispwin -I`.

Display calibration opens a separate console because Argyll's instrument workflow is interactive. Camera profiling uses `QProcess`, merges output channels into the visible log, and sequences `colprof` only after successful chart recognition. Never mark a profile successful solely because a process exited: require the expected ICC file and validate it.

Profile installation is a material external-state change and must remain behind explicit confirmation. Keep generated calibration/profile files in user-selected locations. ArgyllCMS is an external executable dependency and is not bundled by PyInstaller.

SD-card import

SdImportWorker recursively enumerates supported image/video extensions, copies to a user-selected destination, optionally preserves relative card folders, skips identical destination files, and creates unique names for collisions. It emits progress and supports cooperative cancellation. Keep import copy-only unless a separately confirmed move workflow is introduced; a card should never be erased by PhotoLab.

White-balance invariant

rawpy output rendered with camera white balance already has the camera multipliers baked into RGB. `meta["wb_baked"]` prevents `apply_recipe` from multiplying by `camera_whitebalance` a second time. Recipe temperature/tint are creative offsets. Preset parsing must preserve as-shot WB unless a preset explicitly declares absolute white balance. Pipeline reordering requires golden-image and preset/WB regression tests.

Release checklist

1. Review the Git diff and preserve unrelated user changes.
 2. Run the complete test suite with a repository-local temporary directory.
 3. Open representative RGB, RAW, DNG, and embedded-preview-fallback files.
 4. Verify as-shot WB, presets, masks, sidecar reload, SD import, GPS selection/map, and full-resolution export.
 5. Build the portable package and confirm manuals, presets, Lensfun data, and companion applications resolve correctly.
 6. Update both manuals and the roadmap; commit/push only when explicitly requested.
-

Files checklist

File	Layer
main.py	Bootstrap
main_window.py	Application UI / orchestration
widgets.py	Reusable UI controls + canvas
imaging.py	Image science + Recipe
workers.py	Background threads
catalog.py	Library database
focus_stack.py	Focus-stack alignment + fusion engine (no Qt)
panorama.py	OpenCV Stitcher v1 panorama engine (no Qt)
presets.py	Preset I/O
qt_utils.py	Qt image helpers
docs/USER_MANUAL.md	End-user guide
docs/DEVELOPER_MANUAL.md	This document

Conventions

- Normalized geometry (crop, points, gradients, brush dabs) uses **0..1** image coordinates.
- Preview may run on a downsampled copy; export always uses full resolution.
- Prefer `getattr(r, "new_field", default)` in the pipeline so older sidecars still load.

Focus stacking

`focus_stack.py` is a Qt-free engine:

- `align_ecc / align_orb` - registration
- `focus_measure` - Laplacian energy + local variance
- `fuse_depth_map / fuse_weighted / fuse_pyramid / fuse_average`
- `focus_stack(paths, ...)` - load → align to reference → fuse → report

`FocusStackWorker` runs this off the UI thread. PhotoLab opens the output path in Develop with a fresh recipe (stacking is **not** part of `apply_recipe`).

Panorama (OpenCV Stitcher v1)

`panorama.stitch_panorama` loads frames, runs `cv2.Stitcher`, crops empty borders, returns BGR uint8 + report.

Known status codes are mapped to readable errors (`ERR_NEED_MORE_IMGS`, homography failure, etc.).

Same product pattern as HDR / focus stack: **output is a new file**, then opened in Develop. Future work: projection controls, exposure compensation UI, manual control points.

Panorama to Video integration

PhotoLab does **not** reimplement Ken Burns / LUT / ffmpeg logic. `create_pan_video` launches:

```
python pano_video.py --image <current> --output-folder <folder>
```

`pano_video.main()` accepts `--image`, `--output-folder`, `--preset`, `--title`.

`PanoramaToVideoApp.load_image_path` preloads the still and sets the output folder.

`open_audio_editor` similarly spawns `audio_editor.py` as a separate process.

This preserves 100% of the standalone tool features while linking the workflow from Develop.

Resource paths & packaging

`app_paths.py`

Resolves bundled folders for **source** runs and **frozen** executables (`sys._MEIPASS`):

Method	Purpose
<code>app_root()</code>	App / bundle root
<code>docs_dir() / manual_file()</code>	User & developer manuals
<code>plugin_dir() / ensure_plugin_dir()</code>	JSON / XMP presets
<code>list_bundled_presets()</code>	<code>.json</code> / <code>.xmp</code> in <code>plugin/</code>

Folders to ship with the executable

```
docs/ USER_MANUAL.md, DEVELOPER_MANUAL.md
plugin/ *.json, *.xmp develop presets (+ README.md)
```

PyInstaller example:

```
pyinstaller --noconfirm --windowed --name PhotoLab \
--add-data "docs:docs" \
--add-data "plugin:plugin" \
main.py
```

On Windows use `;` instead of `:` in `--add-data`. One-folder builds keep `docs/` and `plugin/` next to the binary; one-file extracts them under `_MEIPASS` (read-only). User-writable presets still go to `~/ .photolab/plugin/`.

Tone / color (#6)

- Parametric + point curves (`curve_points`, `curve_r/g/b_points`)
- Split toning fields on Recipe
- `match_exposure_selected / match_wb_selected` in main window

Detail / WB (#7)

- `output_sharpen_params(ppi, media)` → amount + radius for screen/matte/glossy
- `skin_tone_mask` + `protect_skin` on sharpen and vibrance
- Dual illuminant: `wb_dual`, `temperature2`, `tint2`, `wb_mix` in `apply_white_balance`

Geometry (#8)

- `apply_keystone` - 4-corner perspective from normalized TL,TR,BR,BL
- `detect_horizon_angle` - Hough near-horizontal edges
- `probe_lensfun / try_lensfun_correct` - status + strength blend
- Canvas: `keystone_mode`, `keystoneChanged`

Lensfun local DB

- `app_paths.lensfun_db_paths()` / `primary_lensfun_db()` discover `./lensfun` layouts
- `imaging._open_lensfun_database()` prefers those paths, then system/bundled

Catalog (#9)

- Tables: `collections`, `collection_members`, `virtual_copies`
- Columns: `people`, `content_hash`, `color_label`
- APIs: `create/list collections`, `find_duplicate_groups`, `create_virtual_copy`, `set_people`

Import / culling (#10)

- `catalog.list_importable_files`, `format_import_name`, `import_photos`
- `workers.ImportWorker` background copy/move
- `_CullingDialog` full-screen keyboard culling

Batch (#11)

- `catalog.relocate_image(old, new)` updates images, `collection_members`, `virtual_copies`
- `export_queue.py` - JSON queue under `~/.photolab/export_queue.json`
- `workers.ExportQueueWorker` processes pending jobs with stored recipe dicts
- UI: `apply_preset_to_selected`, `batch_rename_move`, `export_queue_*`

Stack / Pano (#12)

- `focus_stack(..., min_align_score=)` excludes weak alignments; report has scores/excluded/used_indices
- `panorama.match_exposure_wb`, `order_paths_by_capture_time`
- UI report dialogs + `_match_style_from_source`

HDR (#13)

- `detect_exposure_brackets`, `exposure_signature`
- `deghost_stack` median motion blend
- `merge_hdr_mertens` / `merge_hdr_debevec` / `merge_hdr` dispatcher
- `HdrMergeWorker` accepts method, `deghost`, `tonemap`

Pan Video (#14)

- Dark ttk theme in `PanoramaToVideoApp._apply_photolab_theme`
- Progress callback computes ETA from elapsed / fraction
- `PhotoLab.create_pan_video` writes `temp*_photolab_graded.jpg` via `apply_recipe` then launches CLI

Diagnostics (#15)

- `imaging.format_raw_error` - actionable RAW decode messages
- `logging_setup.recent_log_lines / system_info_text / build_problem_report`
- UI: `export_problem_report` from Help and Debug Console